

ML course season 2

Лекция: Снижение размерности — PCA, t-SNE, UMAP

Паша Кашмель

Курс по машинному обучению

7 мая 2026 г.

План лекции

- 1 Зачем снижать размерность?
- 2 PCA: метод главных компонент
- 3 Выбор числа компонент
- 4 Интерпретация PCA: loadings
- 5 t-SNE и UMAP: нелинейная визуализация
- 6 Итоги и практика

Проклятие размерности

Проблема

При большом числе признаков d :

- данные становятся **разреженными** — все точки «далеко» друг от друга
- модели переобучаются или работают медленно
- многие признаки могут быть шумом или дублировать друг друга

Мультиколлинеарность

Если признаки коррелируют между собой, матрица $X^T X$ плохо обусловлена. Линейные модели становятся нестабильными.

Исходное пространство
 $d = 100$ признаков

Сниженная
размерность
 $k \ll d$

Быстрее, лучше
интерпретируется

Четыре причины снижать размерность

1. Визуализация

Человек видит 2D и 3D.

Проекция в 2D позволяет увидеть кластеры, выбросы, структуру.

2. Устранение шума

Малые компоненты дисперсии часто соответствуют шуму, а не сигналу.

Отбрасывая их, мы **регуляризуем** данные.

3. Ускорение обучения

Меньше признаков $\Rightarrow$ меньше параметров модели $\Rightarrow$ быстрее обучение и предсказание.

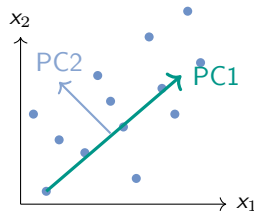
4. Борьба с мультиколлинеарностью

РСА-компоненты **ортогональны** по построению:
корреляция между ними равна нулю.

Задача

Найти новые оси (главные компоненты) такие, что:

- ось 1 — направление **максимальной дисперсии**
- ось 2 — **максимальная дисперсия, ортогонально** оси 1
- и т.д.



Формально

$$\mathbf{z} = \mathbf{W}^T \mathbf{x}, \quad \mathbf{W} \in \mathbb{R}^{d \times k}$$

Столбцы $\mathbf{W}$ — **собственные векторы** матрицы ковариации Σ .

Матрица ковариации и SVD

Шаг 1: стандартизация

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

Обязательно, если признаки имеют разные шкалы!

Шаг 2: матрица ковариации

$$\Sigma = \frac{1}{n-1} \tilde{X}^T \tilde{X} \in \mathbb{R}^{d \times d}$$

Шаг 3: SVD / собственные числа

$$\Sigma = V \Lambda V^T$$

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$$

Берём первые k столбцов V как главные компоненты.

На практике

`sklearn.decomposition.PCA`
использует **SVD** — быстро и численно стабильно.

Объяснённая дисперсия

Доля объяснённой дисперсии

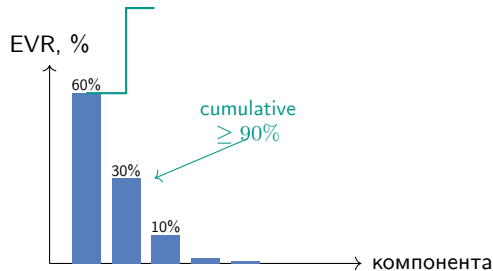
$$\text{EVR}_k = \frac{\lambda_k}{\sum_{j=1}^d \lambda_j}$$

Показывает, **сколько информации** сохраняет k -я компонента.

Суммарная (cumulative)

$$\text{CumEVR}(K) = \frac{\sum_{k=1}^K \lambda_k}{\sum_{j=1}^d \lambda_j}$$

Обычно выбирают K : $\text{CumEVR}(K) \geq 0.90$



```
pca.explained_variance_ratio_
```

Как выбрать k ?

Метод «локтя» (scree plot)

Строим график дисперсии по компонентам.

Выбираем точку излома — после неё прирост мал.

По порогу дисперсии

Задаём порог (90%, 95%) и берём минимальное k , при котором он достигается.

$$\text{CumEVR}(k) \geq \theta$$

Trade-off: качество vs. простота

k	Качество	Сложность
мало	хуже	меньше
много	лучше	больше

Для визуализации

$k = 2$ или $k = 3$ — всегда!

Для модели — подбирать через cross-validation.

Что такое loadings?

Loadings — веса признаков в компонентах

Каждая главная компонента — **линейная комбинация** исходных признаков:

$$z_1 = w_{11}x_1 + w_{21}x_2 + \dots + w_{d1}x_d$$

Коэффициенты w_{j1} — это **loadings** (нагрузки).

Интерпретация

Большой $|w_{j1}| \Rightarrow$ признак x_j **сильно влияет** на PC1.

Знак показывает направление вклада.

Пример: PC1 на Iris

Признак	Loading
sepal length	+0.52
sepal width	−0.27
petal length	+0.58
petal width	+0.56

PC1 $\approx$ «общий размер цветка»

PCA в sklearn: код

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
# Стандартизация обязательна!
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Обучение PCA
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Объяснённая дисперсия
```

```
print(pca.explained_variance_ratio_)
```

```
# Loadings
```

```
print(pca.components_) # (n_comp, n_features)
```

Ключевые атрибуты

Атрибут	Что хранит
.components_	loadings
.explained_	дисперсия
variance_ratio_	доля EVR
.singular_values_	SVD-значения

Важно

Всегда стандартизировать до PCA:
иначе признаки с большим
масштабом **доминируют**.

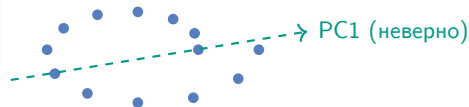
Ограничения PCA

PCA — линейный метод

PCA ищет **линейные** подпространства. Нелинейные структуры (спирали, кольца, «рулетки») теряются.

Пример

Данные лежат на искривлённом многообразии в 3D, но PCA «разворачивает» их неправильно.



Нелинейная структура

t-distributed Stochastic Neighbor Embedding

Сохраняет **локальную структуру**: близкие точки в исходном пространстве остаются близкими в 2D.

Алгоритм (кратко)

- 1 Строим матрицу похожести в оригинальном пространстве p_{ij} (Гаусс)
- 2 Строим матрицу в 2D q_{ij} (t-распределение Стьюдента)
- 3 Минимизируем KL-дивергенцию: $KL(P||Q)$

Важные ограничения t-SNE

- **Нет** обратного преобразования
- Расстояния между кластерами **не интерпретируются**
- Зависит от perplexity
- Медленный на больших данных

UMAP: быстрее и лучше сохраняет структуру

Uniform Manifold Approximation and Projection

Основан на топологии многообразий.
Сохраняет **и локальную, и глобальную** структуру.

Преимущества перед t-SNE

- Значительно **быстрее** на больших данных
- Лучше сохраняет **глобальную структуру**
- Поддерживает **transform()** на новых данных

PCA vs t-SNE vs UMAP

	PCA	t-SNE	UMAP
Линейный	да	нет	нет
Скорость	быстро	медл.	быстро
Transform	да	нет	да
Для моделей	да	нет	осторожно

t-SNE и UMAP — **только для визуализации**,
не для признаков в моделях!

t-SNE и UMAP в коде

```
from sklearn.manifold import TSNE

tsne = TSNE(
    n_components=2,
    perplexity=30,
    random_state=42,
)
X_tsne = tsne.fit_transform(X_scaled)
```

```
import umap

reducer = umap.UMAP(
    n_components=2,
    n_neighbors=15,
    min_dist=0.1,
    random_state=42,
)
X_umap = reducer.fit_transform(X_scaled)
```

Рекомендованный пайплайн

Стандартизация → PCA до 50 компонент (опционально) → t-SNE / UMAP до 2D
Предварительный PCA ускоряет t-SNE и снижает шум.

Когда что использовать

РСА — выбор по умолчанию для препроцессинга

- Много признаков, нужно ускорить обучение
- Нужно убрать мультиколлинеарность
- Нужна **интерпретируемость** компонент
- Нужно объяснить, сколько информации сохранено

t-SNE / UMAP — только визуализация

- Хотим **увидеть** кластеры, классы, структуру
- Нелинейная структура в данных
- **Не использовать** как признаки в моделях

Главная идея

Снижение размерности — это не потеря информации, а её **концентрация**.
Мы отбрасываем шум и оставляем сигнал.

Спасибо за внимание!

Полезные ресурсы

education.yandex.ru/handbook/ml scikit-learn.org
umap-learn.readthedocs.io